

Y

You

O

Only

L

Look

O

Once

Mosharof Hossain

AI Engineer @ Brain Station 23

What is Computer Vision?

Computer Vision is a field of AI that trains machines to **interpret and understand** visual information from images and video — the way humans do naturally.

Human vs Machine

Human Eye

→ **Camera Sensor**

Brain Pattern Match

→ **Neural Network**

"I see a cat"

→ **class: cat, 0.97**

Core CV Tasks



Image Classification

What is in this image?



Object Detection

Where are all the objects?



Segmentation

Pixel-level object masks



Pose Estimation

Human joint keypoints

How Machines See: Images as Numbers



Each pixel = 3 numbers (R, G, B) — 0 to 255

[255, 100, 50] = orange pixel [30, 30, 200]
= blue pixel

Why Detection is Hard

- Objects vary in scale, rotation, lighting
- Occlusion — objects hide behind each other
- Must run in real-time (30+ FPS for video)
- Hundreds of objects per scene

YOLO — You Only Look Once

Traditional detectors scan thousands of region proposals. YOLO treats detection as a **single regression problem** — it looks at the image exactly **once** and predicts all boxes + classes simultaneously.

Two-Stage (R-CNN) — SLOW

1

Region Proposal

Generate 2000+ candidate boxes

2

Feature Extract

Run CNN on each region separately

3

Classify

Classify each region one by one

~7 FPS — too slow for real-time

YOLO — One Stage — FAST

1

Divide into Grid

Split image into $S \times S$ cells

2

Predict All at Once

Each cell predicts boxes + classes

3

Filter Output

NMS removes duplicate boxes

30-200+ FPS — real-time ready!

YOLO Evolution: 2015 → 2026

2015

YOLOv1

Invented the single-pass idea. 45 FPS on Titan X. Redmon & Farhadi, University of Washington.

2018

YOLOv3

Multi-scale predictions, DarkNet-53 backbone. Big leap in small object detection.

2020

YOLOv5

Ultralytics rewrite in PyTorch. Modular, easy to use. Became the most popular version.

2023

YOLOv8

Anchor-free head, decoupled classification/regression. Full task suite: detect, seg, pose, classify.

2024

YOLO11

Efficiency-focused. Lighter with better accuracy. Backbone improvements.

2026

YOLO26

★ CURRENT

NMS-free end-to-end inference. Removes DFL. MuSGD optimizer. Up to 43% faster CPU vs v11.

YOLO Architecture: Backbone → Neck → Head



Backbone

Feature Extractor

Convolutional layers that scan the image and extract hierarchical features — edges at early layers, complex shapes at deeper layers.

e.g. CSPDarkNet / C2f modules

→ Multi-scale feature maps



Neck

Feature Fusion

Combines features from different backbone depths using FPN/PAN paths, so the model can detect both small and large objects.

e.g. PANet / BiFPN

→ Fused multi-scale features



Head

Prediction Layer

Makes final predictions: bounding box coordinates (x, y, w, h), objectness score, and class probabilities for each grid cell.

e.g. Decoupled detect head

→ Boxes + Classes + Scores

Training Loss =

Box Loss

(position/size)

+

Cls Loss

(class probs)

+

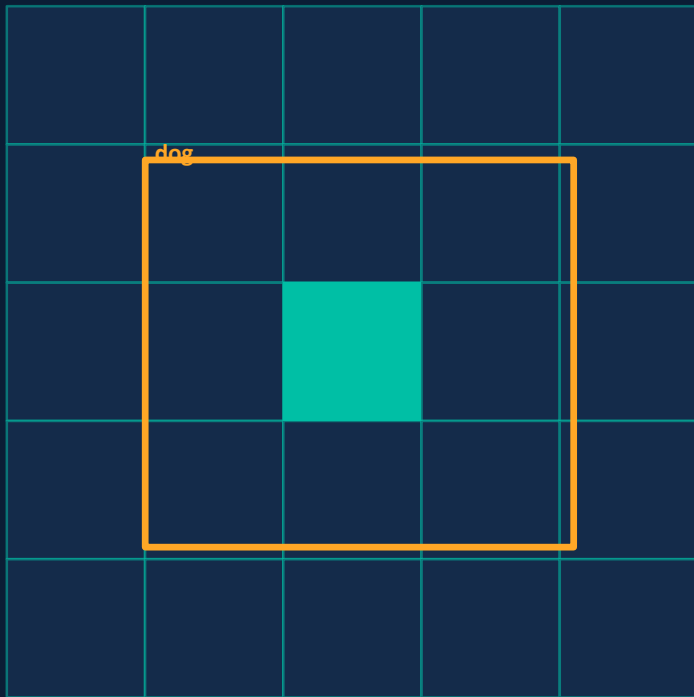
DFL Loss

(distribution focal)

Note: YOLO26 removes DFL — simpler head

How YOLO Makes Predictions: Grid & Bounding Boxes

Grid Cell Prediction



Each active cell predicts a vector:

x, y	w, h	conf	$C_1 \dots C_n$
--------	--------	------	-----------------

Key Concepts

🔪 mAP — Mean Average Precision

The main accuracy metric. Measures how well predicted boxes overlap with ground truth across all classes. Higher = better.

📐 IoU — Intersection over Union

Overlap ratio between predicted and actual box. $\text{IoU} > 0.5$ typically counts as a correct detection.

🔊 NMS — Non-Max Suppression

After prediction, keeps only the highest-confidence box per object, removes duplicates. YOLO26 eliminates this step.

👤📦 Confidence Score

Probability that a box contains an object $\times$ how accurate the box is. Used to filter low-quality detections.

Fine-Tuning: Transfer Learning for Vision

🔗 **Analogy:** A doctor doesn't relearn physics before specializing — they build on existing knowledge. YOLO pretrained on COCO already knows **edges, shapes, textures, and general objects**. Fine-tuning adapts that for your domain.

What COCO Pretrained YOLO Knows

✓ 80 common object classes (person, car, dog, chair...)

✓ Scale-invariant feature detection

✓ Occlusion handling patterns

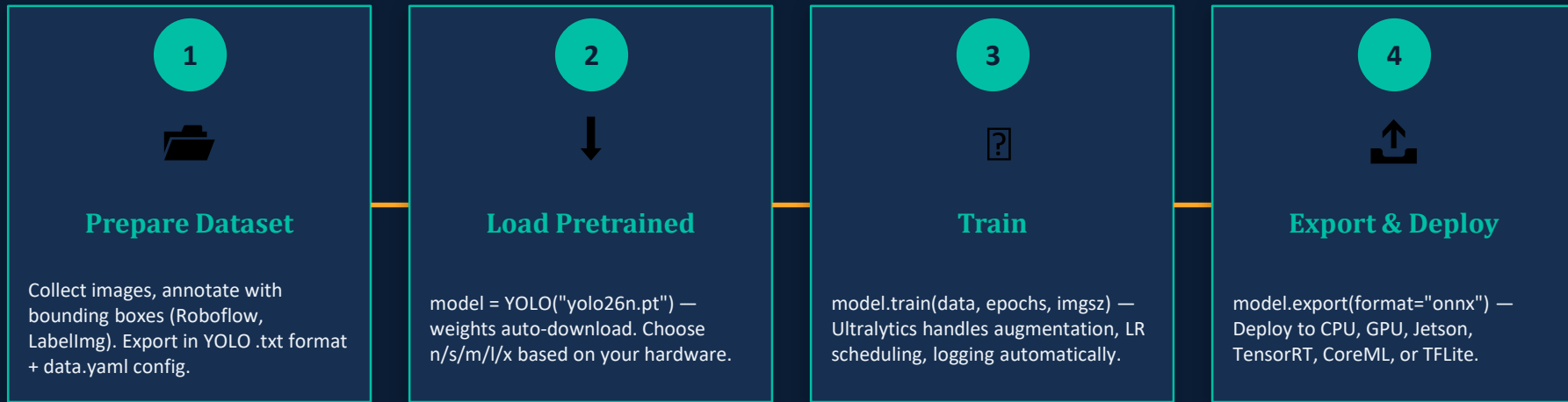
✓ Edge and texture recognition at multiple scales

Training from Scratch vs Fine-Tuning

	From Scratch	Fine-Tune
Data needed	100,000+ images	200-500 images
GPU time	Days / weeks	Minutes / hours
Cost	Very high	Low
Risk	May not converge	Reliable

💡 Layers to freeze: Early backbone layers (general features) → Only train neck + head + last few backbone blocks for custom data

Fine-Tuning with Ultralytics: End-to-End Pipeline



Full Training Script:

```
from ultralytics import YOLO

model = YOLO("yolo26n.pt") # nano: fast. swap n->s->m->l->x for more accuracy

results = model.train(
    data
    ="custom_data.yaml", epochs=100, imgsz=640, batch=16
)
```